

3-DAY MASTER PLAN: all features working

E-Learning Platform. Read this first, then open the day guides. Three days, three modules. Each day ends with an **acceptance checklist**: if every box is ticked, that day's module works and you may start the next one.

0. The honest part (please read)

- **I wrote and tested everything I could here.** The Java logic of OTP, Instructor and Video was compiled with real Java and run with fake repositories (**33 + 49 + 101 checks, 0 failures**). All the new screens were run in a real browser against a pretend backend, with a real playable video (**20 + 31 + 48 checks, 0 failures**).
- **I could not compile or start the real Spring Boot backend** in my environment (Maven downloads are blocked), and I could not call real Twilio, Razorpay, Google or Gemini. So the Spring wiring (annotations, imports, table creation, Security rules) has **never run**. I cannot honestly promise "zero errors". What I can promise is that the logic is tested and that every day has a checkpoint where you find a problem within minutes instead of at the end. If a compiler or start-up message appears, paste it to me; fixes are usually one line.
- The plan is tight: one person doing everything in three days is possible but heavy. **Use the two-person split in section 7.**
- Secrets: the repo is public and `application.properties` has a real-looking database password and (earlier) a Gemini key. **Rotate both**, then use environment variables only (`DB_PASSWORD`, `GEMINI_API_KEY`, `TWILIO_*`, `RAZORPAY_*`). Never commit them.

1. The map

Day	Module	Guides to open (in this order)	You are done when
1	Base + OTP + AI	1. CHANGES-7.pdf (apply all B and F sections) 2. MIDNIGHT-CSS-FINAL.pdf 3. DAY1-OTP-BACKEND.pdf and DAY1-OTP-FRONTEND.pdf 4. AI-BACKEND-GUIDE.pdf and AI-FRONTEND-GUIDE.pdf	register, verify the code, log in; images upload; AI quiz and chat answer
2	Instructor + Google	1. DAY2-INSTRUCTOR-BACKEND.pdf and DAY2-INSTRUCTOR-FRONTEND.pdf 2. DAY2-GOOGLE-OAUTH-BACKEND.pdf and DAY2-GOOGLE-OAUTH-FRONTEND.pdf	instructor applies, admin rejects then approves, instructor does course CRUD; Google login works
3	Videos + Razorpay	1. DAY3-VIDEOS-BACKEND.pdf and DAY3-VIDEOS-FRONTEND.pdf 2. DAY3-PAYMENT-RAZORPAY-BACKEND.pdf and DAY3-PAYMENT-RAZORPAY-FRONTEND.pdf	upload, buy, watch, resume, staff stats; test payment succeeds

Do the days in order. Day 2 needs Day 1's `OtpService`. Day 3 needs Day 2's instructor classes.

Payment extras (course owner's share and invoices). The Razorpay work above is Day 1 of a separate 3-day payment track, `PAYMENTS-3-DAY-PLAN.pdf`. Its Days 2 and 3 (instructor earnings, admin payouts, invoices with a PDF download) come after this plan. The final `SecurityConfig` in section 8 already contains the one extra rule they need.

2. Rules for all three days

1. Make a git branch first: `git checkout -b three-day-build`. Commit at the end of every step, so you can go back.
2. After **every** step, start the backend (`mvn spring-boot:run`) and the frontend (`ng serve --port 8081`) and look. A mistake caught one step later is easy; caught a day later it is a mess.
3. **Order of the Security rules matters** (first match wins). Section 8 gives you the **final complete SecurityConfig.java**. CHANGES-7 replaces that file entirely, so the other guides' one-line additions must be added **after** it. The easy way: copy the final file from section 8 once, on Day 1, and skip every other "add this line to SecurityConfig" instruction in all guides.
4. Same for `login.component.ts`: the Google guide replaces the whole file. Use the final file in section 10, which already contains the Google, OTP and instructor lines.
5. `application.properties`: section 9 lists everything in one place. Add each block once; never duplicate a key.
6. New feature modules go **before** `AppRoutingModule` in `app.module.ts`. The final list is: `imports: [BrowserModule, BrowserAnimationsModule, HttpClientModule, SharedModule, AiModule, VerificationModule, InstructorModule, VideoModule, AppRoutingModule]` (your file may have a few more, keep them).

3. DAY 1: base, OTP and AI

Morning (about 3 h): base 1. Apply `CHANGES-7.pdf`. Backend sections B1 to B6, frontend F1 to F10. When you reach B1 (`SecurityConfig`), use the **final file in section 8 here instead** and skip it later. 2. Apply `MIDNIGHT-CSS-FINAL.pdf` (all the css files). 3. **Checkpoint 1:** backend starts, frontend builds, login works, a course image uploads, the footer shows.

Afternoon (about 3 h): OTP 4. Backend: `DAY1-OTP-BACKEND.pdf` (9 new files, the 3 edits; the Security line is already in the final file). 5. Frontend: `DAY1-OTP-FRONTEND.pdf` (5 new files, 3 edits). 6. Set `OTP_REQUIRED=true` and `OTP_DEV_EXPOSE_CODE=true`. The code appears on the page and in the backend console. **Twilio comes at the end of Day 3** (section 6) so the demo does not depend on it. 7. **Checkpoint 2:** the OTP checklist below.

Evening (about 2 h): AI 8. `AI-BACKEND-GUIDE.pdf` and `AI-FRONTEND-GUIDE.pdf`. Set `GEMINI_API_KEY`. (The `/api/ai/**` rule is already in the final `SecurityConfig`.) 9. **Checkpoint 3:** the AI checklist.

Day 1 acceptance checklist

- [] Backend starts with no red errors; Swagger opens
- [] Old accounts (including the seeded admin) still log in with OTP switched **on**
- [] New customer registers, lands on the verify page, wrong code shows "tries left", right code works, login works
- [] A new customer who skips verification gets sent to the verify page when logging in
- [] With `OTP_REQUIRED=false` registration still ends on the login page
- [] Admin can browse and upload a course image; the footer shows on every page

- ☐ AI quiz creates questions and marks them (or shows "AI is not switched on" without a key); chatbot button appears for customers only

4. DAY 2: instructors and Google

Morning (about 4 h): instructor module 1. Backend: `DAY2-INSTRUCTOR-BACKEND.pdf` (10 new files). 2. Frontend: `DAY2-INSTRUCTOR-FRONTEND.pdf` (12 new files, 5 edits; for `login.component.ts` use section 10 if you do Google today). 3. **Checkpoint 4:** walk the whole story once: apply, reject with a reason, see the reason, resubmit, approve, add/edit/delete a course.

Afternoon (about 3 h): Google login 4. Backend: `DAY2-GOOGLE-OAUTH-BACKEND.pdf`. Frontend: `DAY2-GOOGLE-OAUTH-FRONTEND.pdf` (it contains the final `login.component.ts`, same as section 10 here). The SecurityConfig step is already done by the final file. 5. Google users new to the system have no pending OTP row, so they count as verified (Google already verified the email). But an existing account that is still waiting for its OTP is **not** allowed in through Google: it gets 403 and is sent to the verify-phone page. 6. **Checkpoint 5:** the Google checklist.

Day 2 acceptance checklist

- ☐ Home page has **Become an instructor**; the form validates every field
- ☐ After submitting, the instructor can log in and sees the status tracker (Submitted, Under review)
- ☐ The admin sees the application under **Instructor Requests**; Reject needs a reason of 5+ characters
- ☐ The instructor sees the reason, edits, resubmits (back to pending)
- ☐ After approval: the green note shows, and **My courses** lets the instructor add, edit and delete
- ☐ An instructor cannot open the admin pages, and cannot edit another instructor's course (403/404)
- ☐ Google login works for a customer (and an admin Google account is refused, as in the guide)

5. DAY 3: videos and Razorpay

Morning (about 4 h): videos 1. Backend: `DAY3-VIDEOS-BACKEND.pdf` (12 new files, edits 4b and 4c; 4a is in the final SecurityConfig). Change the two multipart limits, **do not duplicate them**. 2. Frontend: `DAY3-VIDEOS-FRONTEND.pdf` (12 new files, 3 edits). 3. Test with a short MP4 (H.264 + AAC) of a few MB. 4. **Checkpoint 6:** the video checklist below.

Afternoon (about 3 h): Razorpay 5. Backend: `DAY3-PAYMENT-RAZORPAY-BACKEND.pdf`. Frontend: `DAY3-PAYMENT-RAZORPAY-FRONTEND.pdf`. Use test keys only. The SecurityConfig step is already done. 6.

Checkpoint 7: pay with the Razorpay test card, the course appears in My Enrollments, and then in **My Learning** if it has videos.

Last hour: Twilio and rehearsal (section 6), then the full demo run in section 7.

Day 3 acceptance checklist

- ☐ Admin uploads an MP4 to a course; the progress bar fills; it appears in the list
- ☐ A `.txt` renamed to `.mp4` is refused and leaves no file in `video-files`
- ☐ A customer who bought the course sees it in **My Learning** and can play it and seek

- [] Stop at 10 s, reload: **"Continuing from 0:10"**
- [] Reach 90%: **Completed**; the admin's table shows Started 1, Finished 1, Average 100%
- [] A customer who did not buy the course is told to enroll
- [] An instructor uploads only to their own course
- [] Razorpay test payment enrolls the customer

6. Twilio, step by step (do this last on Day 3, or earlier if you have time)

The code is finished; Twilio is only configuration, but it is the one part I could not test, so leave time.

1. Sign up at [twilio.com](https://www.twilio.com) (free trial) and verify your phone.
2. Console: get a trial phone number. Copy **Account SID** and **Auth Token**.
3. **Trial limit**: you can only text numbers listed under *Verified Caller IDs*. Add every phone that will be used in the demo (yours, your teammates', your SME's).
4. Set (PowerShell): `$env:OTP_PROVIDER="twilio"`, `$env:OTP_REQUIRED="true"`,
`$env:OTP_DEV_EXPOSE_CODE="false"`, `$env:OTP_COUNTRY_CODE="+91"`,
`$env:TWILIO_ACCOUNT_SID="AC..."`, `$env:TWILIO_AUTH_TOKEN="..."`,
`$env:TWILIO_FROM_NUMBER="+1..."`.
5. Restart, register with a verified number, and the SMS arrives.
6. **If SMS does not arrive** (Indian sender rules, trial limits): switch back to `OTP_PROVIDER=dev` and `OTP_DEV_EXPOSE_CODE=true` for the demo. The flow is identical. Say honestly that Twilio delivery depends on the account.
7. Rotate the Auth Token if it ever appears in a chat or screenshot.

7. Two-person split (recommended)

	Person A (backend)	Person B (frontend)
Day 1	CHANGES-7 backend, Day 1 OTP backend, AI backend	CHANGES-7 frontend, Midnight CSS, Day 1 OTP frontend, AI frontend
Day 2	Day 2 instructor backend, Google backend	Day 2 instructor frontend, Google frontend
Day 3	Day 3 videos backend, Razorpay backend, Twilio	Day 3 videos frontend, Razorpay frontend, demo script

Sync point each day: the checklist, done together at the end.

Final demo run (30 minutes, Day 3 end): register customer (verify OTP) then apply as instructor (verify OTP) then admin rejects with a reason then instructor resubmits then admin approves then instructor creates a course and uploads a video then customer buys it (Razorpay test) then watches 10 s, reloads, resumes then admin opens the statistics.

8. FINAL `SecurityConfig.java` (replaces the file from CHANGES-7 B1)

This is CHANGES-7's B1 file with every other guide's rule inserted in the right place. **Use this file and skip all other `SecurityConfig` instructions.** The lines added by the new guides are commented with the

day.

```
package com.examly.springapp.config;

import com.examly.springapp.repository.UserRepo;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.security.web.header.writers.ReferrerPolicyHeaderWriter;

/**
 * Who may call which endpoint. Two filter chains exist:
 * <ol>
 * <li>Swagger/OpenAPI documentation paths (public, with a relaxed
 * Content-Security-Policy so the UI can load)</li>
 * <li>The API itself (JWT, roles, strict "default-src 'none'" policy)</li>
 * </ol>
 * In the API chain the ORDER of the requestMatchers matters: the first match
 * wins, so specific rules come first.
 *
 * @author Suriya
 */
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    /** Paths served by springdoc: the OpenAPI JSON/YAML and the Swagger UI. */
    private static final String[] SWAGGER_PATHS = {
        "/swagger-ui/**", "/swagger-ui.html", "/v3/api-docs", "/v3/api-docs/**", "/v3/api-docs.yaml"
    };

    /**
     * Swagger UI needs its own scripts, styles and inline styles; this is only used
     * for the documentation paths.
     */
    private static final String SWAGGER_CSP = "default-src 'self'; script-src 'self' 'unsafe-inline'; "
        + "style-src 'self' 'unsafe-inline'; "
        + "img-src 'self' data:; font-src 'self' data:; frame-ancestors 'none'";

    private final JwtUtil jwtUtil;
    private final UserRepo userRepo;
    private final RateLimiter rateLimiter;
    private final int globalPerMinute;
    private final int authPerMinute;

    public SecurityConfig(JwtUtil jwtUtil, UserRepo userRepo, RateLimiter rateLimiter,
        @Value("${ratelimit.global.per-minute:120}") int globalPerMinute,
        @Value("${ratelimit.auth.per-minute:10}") int authPerMinute) {
        this.jwtUtil = jwtUtil;
        this.userRepo = userRepo;
        this.rateLimiter = rateLimiter;
        this.globalPerMinute = globalPerMinute;
        this.authPerMinute = authPerMinute;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);
    }
}
```

```

/**
 * Chain for the API documentation. It is matched first (Order 1) and only for
 * the Swagger paths, permits everyone and relaxes the Content-Security-Policy
 * for those paths only. The request-id and rate-limit filters still run, so the
 * docs cannot be used to flood the server.
 */
@Bean
@Order(1)
public SecurityFilterChain swaggerFilterChain(HttpSecurity http) throws Exception {
    http
        .securityMatcher(SWAGGER_PATHS)
        .csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .headers(h -> h
            .contentSecurityPolicy(c -> c.policyDirectives(SWAGGER_CSP))
            .referrerPolicy(r ->
r.policy(ReferrerPolicyHeaderWriter.ReferrerPolicy.NO_REFERRER)))
        .authorizeHttpRequests(auth -> auth.anyRequest().permitAll());
    http.addFilterBefore(new RateLimitFilter(rateLimiter, globalPerMinute, authPerMinute),
        UsernamePasswordAuthenticationFilter.class);
    http.addFilterBefore(new RequestIdFilter(), RateLimitFilter.class);
    return http.build();
}

@Bean
@Order(2)
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    RestSecurityHandlers handlers = new RestSecurityHandlers();
    http
        // stateless API: no cookies authenticate API calls, only the Bearer token
        .csrf(AbstractHttpConfigurer::disable)
        .cors(Customizer.withDefaults())
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .headers(h -> h
            .contentSecurityPolicy(c -> c.policyDirectives("default-src 'none'; frame-ancestors
'none'"))
            .referrerPolicy(r ->
r.policy(ReferrerPolicyHeaderWriter.ReferrerPolicy.NO_REFERRER)))
        .exceptionHandling(e -> e.authenticationEntryPoint(handlers).accessDeniedHandler(handlers))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.OPTIONS, "/*").permitAll()
            .requestMatchers("/api/register", "/api/login", "/api/refresh", "/api/logout",
"/api/health", "/api/auth/google") // Google added (Day 2)
            .permitAll()

            // Day 1: mobile OTP (the person is not logged in yet when they verify)
            .requestMatchers("/api/otp/*").permitAll()
            // Day 2: anyone may apply to become an instructor
            .requestMatchers("/api/instructor/register").permitAll()
            // Day 3: the video bytes; the signed ticket in the address is checked inside the
controller
            .requestMatchers(HttpMethod.GET, "/api/video/stream/*").permitAll()

            // API documentation (normally served by the swagger chain above; listed here
            // too so it is never blocked)
            .requestMatchers("/swagger-ui/*", "/swagger-ui.html", "/v3/api-docs", "/v3/api-
docs/*",
                "/v3/api-docs.yaml")
            .permitAll()

            // uploaded course images are public so the browser can show them in an <img> tag
(read only)
            .requestMatchers(HttpMethod.GET, "/uploads/*").permitAll()

            // course image upload: admin only
            .requestMatchers(HttpMethod.POST, "/api/upload/*").hasAuthority("ADMIN")

            // admin management: only an existing admin can list, add or remove admins
            .requestMatchers("/api/admin/*").hasAuthority("ADMIN")

            // Day 2: instructors (the approval check happens inside the service)

```

```

        .requestMatchers("/api/instructor/**").hasAuthority("INSTRUCTOR")

// Day 1: AI quiz and chatbot (customers only)
        .requestMatchers("/api/ai/**").hasAuthority("CUSTOMER")

// Day 3: Razorpay payments (customers only)
        .requestMatchers("/api/payment/**").hasAuthority("CUSTOMER")

// Payment track, Day 3: invoices (a customer sees his own, an admin sees all)
        .requestMatchers("/api/invoice/**").hasAnyAuthority("CUSTOMER", "ADMIN")

// Day 3: videos. The more specific rules come first.
        .requestMatchers(HttpMethod.GET, "/api/video/my-learning").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.PUT, "/api/video/*/progress").hasAuthority("CUSTOMER")
        .requestMatchers("/api/video/**").hasAnyAuthority("ADMIN", "INSTRUCTOR",
"CUSTOMER")

// AI search: logged-in customers only (usage is limited per user)
        .requestMatchers("/api/course-ai/search").hasAuthority("CUSTOMER")

// courses ("/api/course/search" is listed first for clarity; the general GET
// rule below also covers it)
        .requestMatchers(HttpMethod.GET, "/api/course/search").hasAnyAuthority("ADMIN",
"CUSTOMER")

        .requestMatchers(HttpMethod.GET, "/api/course", "/api/course/**")
        .hasAnyAuthority("ADMIN", "CUSTOMER")
        .requestMatchers("/api/course", "/api/course/**").hasAuthority("ADMIN")

// customers (ownership is checked again inside the controller)
        .requestMatchers(HttpMethod.POST, "/api/customer").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.PUT, "/api/customer/**").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/customer/user/**").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/customer/**").hasAuthority("ADMIN")

// orders
        .requestMatchers(HttpMethod.POST, "/api/order").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/order").hasAuthority("ADMIN")
        .requestMatchers(HttpMethod.GET, "/api/order/customer/**").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/order/**").hasAnyAuthority("ADMIN",
"CUSTOMER")

        .requestMatchers(HttpMethod.DELETE, "/api/order/**").hasAuthority("CUSTOMER")

// reviews
        .requestMatchers(HttpMethod.POST, "/api/review").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/review/user/**").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/review/summary", "/api/review/course/**")
        .hasAnyAuthority("ADMIN", "CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/review", "/api/review/**")
        .hasAnyAuthority("ADMIN", "CUSTOMER")
// a customer may delete only their own review (ownership is checked in the
controller);

// this rule must stay ABOVE the admin rule below because the first matching rule
wins

        .requestMatchers(HttpMethod.DELETE, "/api/review/my/**").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.DELETE, "/api/review/**").hasAuthority("ADMIN")

// dashboards: customers see their own numbers (checked again in the
// controller), admins see the platform
        .requestMatchers(HttpMethod.GET, "/api/dashboard/admin").hasAuthority("ADMIN")
        .requestMatchers(HttpMethod.GET,
"/api/dashboard/customer/**").hasAuthority("CUSTOMER")

// carts
        .requestMatchers(HttpMethod.GET, "/api/cart").hasAuthority("ADMIN")
        .requestMatchers("/api/cart", "/api/cart/**").hasAuthority("CUSTOMER")

        .anyRequest().authenticated();

// filter order: request id -> rate limit -> JWT
http.addFilterBefore(new RateLimitFilter(rateLimiter, globalPerMinute, authPerMinute),
UsernamePasswordAuthenticationFilter.class);

```

```

        http.addFilterBefore(new RequestIdFilter(), RateLimitFilter.class);
        http.addFilterAfter(new JwtRequestFilter(jwtUtil, userRepo), RateLimitFilter.class);
        return http.build();
    }
}

```

B2. OrderServiceImpl.java

Changes: imports `DuplicateResourceException`, `HashSet`, `Set`, and the lines marked `// NEW` inside `addOrder`. A repeated course gives 409 with the message "You are already enrolled in ".

springapp/src/main/java/com/examly/springapp/service/OrderServiceImpl.java

```

## 9. FINAL `application.properties` additions (add once each)

````properties
--- from CHANGES-7 (Day 1); the two multipart lines are CHANGED on Day 3 ---
app.upload.dir=${UPLOAD_DIR:uploads}
spring.servlet.multipart.max-file-size=200MB
spring.servlet.multipart.max-request-size=201MB

--- Mobile OTP (Day 1) ---
otp.required=${OTP_REQUIRED:false}
otp.provider=${OTP_PROVIDER:dev}
otp.dev-expose-code=${OTP_DEV_EXPOSE_CODE:false}
otp.country-code=${OTP_COUNTRY_CODE:+91}
twilio.account-sid=${TWILIO_ACCOUNT_SID:}
twilio.auth-token=${TWILIO_AUTH_TOKEN:}
twilio.from-number=${TWILIO_FROM_NUMBER:}

--- Google login (Day 2) ---
google.client-id=${GOOGLE_CLIENT_ID:}

--- Videos (Day 3) ---
video.storage-dir=${VIDEO_DIR:video-files}
video.max-size-mb=${VIDEO_MAX_MB:200}
video.ticket-minutes=${VIDEO_TICKET_MINUTES:120}

--- Razorpay (Day 3) ---
razorpay.key-id=${RAZORPAY_KEY_ID:}
razorpay.key-secret=${RAZORPAY_KEY_SECRET:}
razorpay.enforce=${RAZORPAY_ENFORCE:false}

```

`.gitignore` additions: `uploads/`, `video-files/`.

## 10. FINAL `login.component.ts` (Google + OTP + instructor)

Use this **whole file** in step 6.5 of the Google guide (it replaces the guide's file). It is the Google guide's file with three additions: the instructor redirect (twice) and the "verify your mobile number" redirect. The `login.component.html` and `.css` stay as the Google guide says.

```

import { AfterViewInit, Component, ElementRef, NgZone, ViewChild } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService } from '../../services/auth.service';
import { CustomerService } from '../../services/customer.service';
import { GoogleAuthService } from '../../services/google-auth.service';

@Component({

```

```

selector: 'app-login',
templateUrl: './login.component.html',
styleUrls: ['./login.component.css']
})
export class LoginComponent implements AfterViewInit {
 loginForm: FormGroup;
 errorMessage = '';
 loading = false;
 showPassword = false;
 /** True when a Google client id is configured, so the Google button is shown. */
 googleEnabled = false;
 @ViewChild('googleButton') googleButton?: ElementRef<HTMLElement>;

 constructor(
 private fb: FormBuilder,
 private auth: AuthService,
 private customerService: CustomerService,
 private router: Router,
 private google: GoogleAuthService,
 private zone: NgZone
) {
 this.googleEnabled = this.google.enabled;
 this.loginForm = this.fb.group({
 email: ['', [Validators.required, Validators.email]],
 password: ['', [Validators.required]]
 });
 }

 get email() { return this.loginForm.get('email'); }
 get password() { return this.loginForm.get('password'); }

 onSubmit(): void {
 if (this.loginForm.invalid || this.loading) { return; }
 this.loading = true;
 this.errorMessage = '';
 this.auth.login(this.loginForm.value).subscribe({
 next: (res: any) => {
 if (res.role === 'INSTRUCTOR') {
 this.loading = false;
 this.router.navigate(['/instructor-register']);
 return;
 }
 if (res.role === 'ADMIN') {
 this.loading = false;
 this.router.navigate(['/dashboard']);
 return;
 }
 // customers must have filled in their details once before they can use the app
 this.customerService.viewCustomerByUserId().subscribe({
 next: () => { this.loading = false; this.router.navigate(['/dashboard']); },
 error: (err) => {
 this.loading = false;
 if (err.status === 404) {
 this.router.navigate(['/customerdashboard']);
 } else {
 this.errorMessage = 'Could not load your profile. Please try again.';
 }
 }
 });
 },
 error: (err) => {
 this.loading = false;
 if (err.status === 401) {
 this.errorMessage = 'Invalid email or password';
 } else if (err.status === 403 && String(err.error?.message || '').includes('verify your mobile number')) {
 this.router.navigate(['/verify-phone'], { queryParams: { email: this.loginForm.value.email } });
 } else if (err.status === 429) {
 this.errorMessage = 'Too many attempts. Please wait a while and try again.';
 } else {
 this.errorMessage = 'Unable to reach the server. Please try again later.';
 }
 }
 });
 }
}

```

```

 }
 }
});
}

// ----- Login with Google (OAuth 2.0) -----

ngAfterViewInit(): void {
 if (this.googleEnabled && this.googleButton) {
 this.google.renderButton(this.googleButton.nativeElement, (credential) =>
this.onGoogleCredential(credential))
 .catch(() => { this.errorMessage = 'Google sign-in could not be loaded. Check your internet
connection.'; });
 }
}

/** Reads the e-mail out of Google's ID token (a JWT) only to pre-fill the verify page. The server never
trusts this. */
private emailFromGoogleCredential(credential: string): string {
 try {
 const part = credential.split('.')[1].replace(/-/g, '+').replace(/_/g, '/');
 const email = JSON.parse(atob(part)).email;
 return typeof email === 'string' ? email : '';
 } catch {
 return '';
 }
}

/** Google calls this outside Angular, so we step back into the Angular zone first. */
private onGoogleCredential(credential: string): void {
 this.zone.run(() => {
 if (this.loading) { return; }
 this.loading = true;
 this.errorMessage = '';
 this.auth.loginWithGoogle(credential).subscribe({
 next: (res: any) => this.goAfterLogin(res),
 error: (err) => {
 this.loading = false;
 if (err.status === 401) {
 this.errorMessage = 'Google sign-in failed. Please try again.';
 } else if (err.status === 403 && String(err.error?.message || '').includes('verify your mobile
number')) {
 // registered with the form earlier and never verified the phone: finish that first
 const email = this.emailFromGoogleCredential(credential);
 if (email) {
 this.router.navigate(['/verify-phone'], { queryParams: { email } });
 } else {
 this.errorMessage = 'Please verify your mobile number first. Log in with your email and
password to start.';
 }
 } else if (err.status === 403) {
 this.errorMessage = 'Admin accounts must log in with their email and password.';
 } else if (err.status === 400) {
 this.errorMessage = 'Login with Google is not set up on the server yet.';
 } else {
 this.errorMessage = 'Unable to reach the server. Please try again later.';
 }
 }
 });
 });
}

/** Same rule as the normal login: admins go to the dashboard, customers need their details saved once.
*/
private goAfterLogin(res: any): void {
 if (res.role === 'INSTRUCTOR') {
 this.loading = false;
 this.router.navigate(['/instructor-register']);
 return;
 }
 if (res.role === 'ADMIN') {

```

```

 this.loading = false;
 this.router.navigate(['/dashboard']);
 return;
 }
 this.customerService.viewCustomerByUserId().subscribe({
 next: () => { this.loading = false; this.router.navigate(['/dashboard']); },
 error: (err) => {
 this.loading = false;
 if (err.status === 404) {
 this.router.navigate(['/customerdashboard']);
 } else {
 this.errorMessage = 'Could not load your profile. Please try again.';
 }
 }
 });
}
}
}
}

```

## 11. If something goes wrong (quick table)

Symptom	First thing to check
Backend will not compile	read the <b>first</b> error only; usually a missing import or a file in the wrong folder
<code>NoSuchBeanDefinitionException</code> at start-up	a file from an earlier day is missing (days depend on each other)
Circular dependency at start-up	use the Day 2 <code>InstructorCourseService</code> exactly as printed
A new page shows the home page	its module is after <code>AppRoutingModule</code>
403 on a new endpoint	the Security rule is missing or below a stricter rule; use section 8
500 on video upload	<code>multipart</code> limits still 20MB, or the folder is not writable
Frontend build error <code>API.X does not exist</code>	a <code>constant.ts</code> addition was missed
An old test breaks	switch <code>OTP_REQUIRED=false</code> ; with it off, behaviour is unchanged

## 12. Known limits (honest list)

- Never compiled or started against real Spring/MySQL here; Twilio, Razorpay, Google and Gemini never called for real.
- Videos: MP4 and WebM only, up to 200 MB by default; no transcoding, no subtitles, no per-video ordering screen (lessons play in upload order).
- If an **admin** deletes an instructor's course through the old admin screens, that course's videos stay on disk (the instructor's own delete cleans up).
- Quiz answer keys live in memory and are lost when the backend restarts (OTP data is in the database).
- The bundle is just over Angular's 1 MB *warning* budget (limit for errors is 2 MB); it builds fine.
- If a hidden test expects registration to navigate straight to `/login`, run with `OTP_REQUIRED=false` (the verify page then forwards to login).